RDMS

Saif Bashar
Lecturer
Department of CSE(UGV)

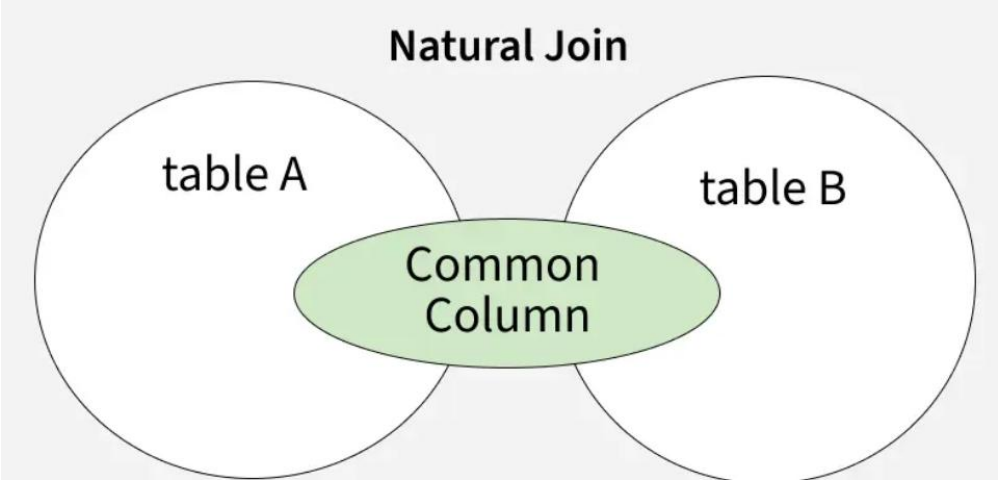
Outlines

Joined Relations

- **Join operations** take two relations and return as a result another relation.
- A join operation is a Cartesian product which requires that tuples in the two relations match (under some condition). It also specifies the attributes that are present in the result of the join
- The join operations are typically used as subquery expressions in the **from** clause
- Three types of joins:
  - Natural join
  - Inner join
  - Outer join

Natural Join in SQL

- Natural join matches tuples with the same values for all common attributes, and retains only one copy of each common column.
- List the names of instructors along with the course ID of the courses that they taught
  - **select** *name*, *course_id*
    from *students, takes*
    where *student.ID = takes.ID*;
- Same query in SQL with "natural join" construct
  - **select** *name*, *course_id*
    from *student* **natural join** *takes*;



Natural Join in SQL

department Table

DEPT_NAME	MANAGER_NAME
IT	ROHAN
SALES	RAHUL
HR	TANMAY
FINANCE	ASHISH
MARKETING	SAMAY

employee Table

EMP_ID	EMP_NAME	DEPT_NAME
1	SUMIT	HR
2	JOEL	IT
3	BISWA	MARKETING
4	VAIBHAV	IT
5	SAGAR	SALES

Natural Join in SQL

department Table

DEPT_NAME	MANAGER_NAME
IT	ROHAN
SALES	RAHUL
HR	TANMAY
FINANCE	ASHISH
MARKETING	SAMAY

employee Table

EMP_ID	EMP_NAME	DEPT_NAME
1	SUMIT	HR
2	JOEL	IT
3	BISWA	MARKETING
4	VAIBHAV	IT
5	SAGAR	SALES

A Cartesian product in a relational database (RDBMS) is a result of combining every row from one table with every row from another table, essentially a cross join without any join conditions.

```
SELECT *
FROM employee , department
```

Natural Join in SQL

EMP_ID	EMP_NAME	DEPT_NAME (Emp)	DEPT_NAME (Manager)	MANAGER_NAME
1	SUMIT	HR	IT	ROHAN
1	SUMIT	HR	SALES	RAJUL
1	SUMIT	HR	HR	TANMAY
1	SUMIT	HR	FINANCE	ASHISH
1	SUMIT	HR	MARKETING	SAMAY
2	JOEL	IT	IT	ROHAN
2	JOEL	IT	SALES	RAJUL
2	JOEL	IT	HR	TANMAY
2	JOEL	IT	FINANCE	ASHISH
2	JOEL	IT	MARKETING	SAMAY
3	BISWA	MARKETING	IT	ROHAN
3	BISWA	MARKETING	SALES	RAJUL
3	BISWA	MARKETING	HR	TANMAY
3	BISWA	MARKETING	FINANCE	ASHISH
3	BISWA	MARKETING	MARKETING	SAMAY

Natural Join in SQL

EMP_ID	EMP_NAME	DEPT_NAME (Emp)	DEPT_NAME (Manager)	MANAGER_NAME
4	VAIBHAV	IT	IT	ROHAN
4	VAIBHAV	IT	SALES	RAJUL
4	VAIBHAV	IT	HR	TANMAY
4	VAIBHAV	IT	FINANCE	ASHISH
4	VAIBHAV	IT	MARKETING	SAMAY
5	SAGAR	SALES	IT	ROHAN
5	SAGAR	SALES	SALES	RAJUL
5	SAGAR	SALES	HR	TANMAY
5	SAGAR	SALES	FINANCE	ASHISH
5	SAGAR	SALES	MARKETING	SAMAY

Natural Join in SQL

department Table

DEPT_NAME	MANAGER_NAME
IT	ROHAN
SALES	RAHUL
HR	TANMAY
FINANCE	ASHISH
MARKETING	SAMAY

employee Table

EMP_ID	EMP_NAME	DEPT_NAME
1	SUMIT	HR
2	JOEL	IT
3	BISWA	MARKETING
4	VAIBHAV	IT
5	SAGAR	SALES

```
SELECT *
FROM employee
NATURAL JOIN department;
```

EMP_ID	EMP_NAME	DEPT_NAME	MANAGER_NAME
1	SUMIT	HR	TANMAY
2	JOEL	IT	ROHAN
3	BISWA	MARKETING	SAMAY
4	VAIBHAV	IT	ROHAN
5	SAGAR	SALES	RAHUL

Natural Join in SQL

- The **from** clause can have multiple relations combined using natural join:

  select $A_1, A_2, \ldots A_n$
  from r_1 **natural join** r_2 **natural join .. natural join** r_n
  where P ;

Join Condition

- The **on** condition allows a general predicate over the relations being joined
- This predicate is written like a **where** clause predicate except for the use of the keyword **on**
- Query example

  select *
  from *student* **join** *takes* **on** *student_ID = takes_ID*

  - The **on** condition above specifies that a tuple from *student* matches a tuple from *takes* if their *ID* values are equal.
- Equivalent to:

  select *
  from *student , takes*
  where *student_ID = takes_ID*

Join Condition

ON Clause
Used **with JOINs** (INNER JOIN, LEFT JOIN, etc.).
Specifies **how two tables are joined**, i.e., the **join condition**.
Determines **which rows from each table are combined**.
Example:

```
SELECT *
FROM Students s
INNER JOIN Enrollments e
ON s.student_id = e.student_id;
```

Only rows where student_id matches in both tables are joined.
If you use LEFT JOIN, the ON clause still decides which rows match, but unmatched rows from the left table will appear with NULLs for the right table.

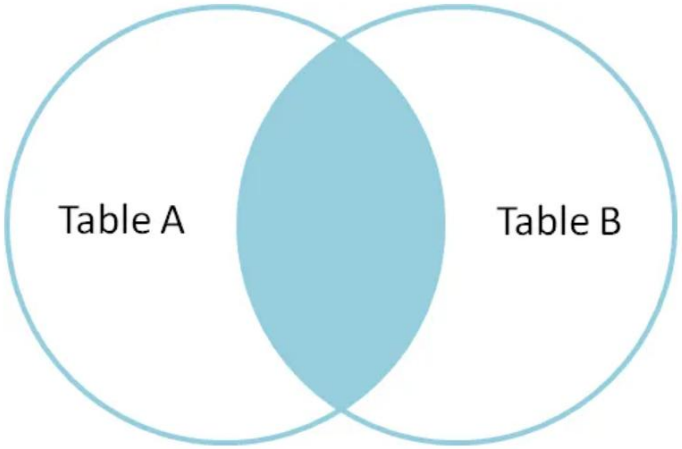
WHERE Clause
Filters rows **after the JOIN has occurred** (or from a single table if no join).
Acts like a **post-processing filter**.
Example:

```
SELECT *
FROM Students s
INNER JOIN Enrollments e
ON s.student_id = e.student_id
WHERE e.grade = 'A';
```

First, tables are joined on student_id.
Then, **only rows with grade = 'A'** are kept in the result.

Inner Join

•The INNER JOIN keyword selects all rows from both tables **when the join condition is met**.
•It creates a result set by **combining rows from both tables** where the specified condition is satisfied.
•The condition typically involves a **common field with matching values** in both tables.
•Only rows with **matching values in the common column** are included in the result.
• JOIN by default is treated as **INNER JOIN**.



Inner Join

Inner Join

Student

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4

```
SELECT StudentCourse.COURSE_ID, Student.NAME, Student.AGE FROM
Student INNER JOIN StudentCourse
ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

Inner Join

Student

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4

```
SELECT StudentCourse.COURSE_ID, Student.NAME,
Student.AGE FROM Student INNER JOIN StudentCourse
ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

COURSE_ID	NAME	AGE
1	HARSH	18
2	PRATIK	19
2	RIYANKA	20
3	DEEP	18

Difference Between Natural Join and Inner Join

Feature	**Natural Join**	**Inner Join**
Definition	Joins tables based on **all columns with the same name and compatible data types**.	Joins tables based on a **specified condition**, usually using ON or USING.
Condition	Automatically uses **common column(s)** between tables.	Requires an **explicit join condition** to be defined.
Syntax Simplicity	Simpler – no need to specify join condition.	More flexible but requires join condition to be written.
Control	Less control – it joins on **all matching column names**, which can be risky.	More control – you choose which columns to join on.
Column Duplication	Removes duplicate columns from the result.	Keeps both columns unless explicitly excluded.
Example	SELECT * FROM A NATURAL JOIN B;	SELECT * FROM A INNER JOIN B ON A.id = B.id;

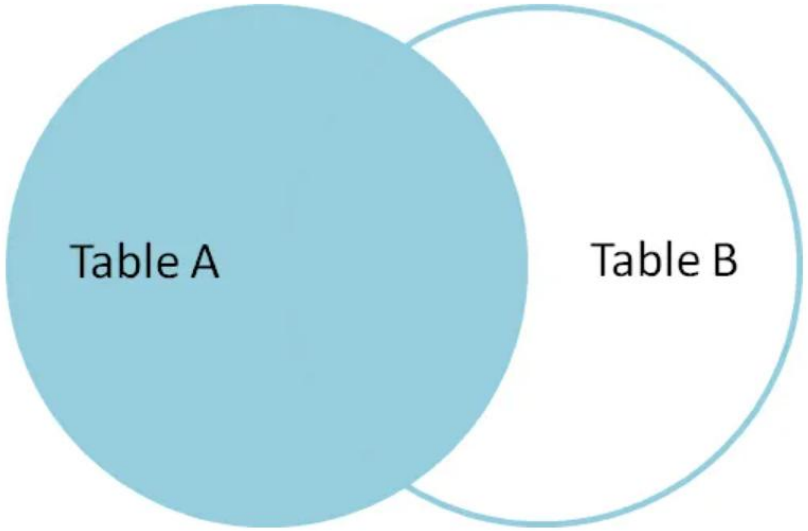
Outer Join

An **Outer Join** is a type of join that returns **matched rows plus unmatched rows** from one or both tables, depending on the type of outer join used.

LEFT OUTER JOIN (or just **LEFT JOIN**)	Returns **all rows from the left table**, and the **matched rows from the right table**. If there's no match, NULL is returned for right table columns.
RIGHT OUTER JOIN (or just **RIGHT JOIN**)	Returns **all rows from the right table**, and the **matched rows from the left table**. If there's no match, NULL is returned for left table columns.
FULL OUTER JOIN	Returns **all rows when there is a match in either table**. Rows with no match in one of the tables will have NULL for the other table's columns.

SQL LEFT JOIN

A LEFT JOIN returns all rows from the left table, along with matching rows from the right table. If there is no match, NULL values are returned for columns from the right table. LEFT JOIN is also known as LEFT OUTER JOIN.



Left JOIN

SQL LEFT JOIN

Student

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18
5	RAVI	DELHI	XXXXXXXXX	19

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4

```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
LEFT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

SQL LEFT JOIN

Student

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	HARSH	DELHI	XXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXX	18
5	RAVI	DELHI	XXXXXXXX	19

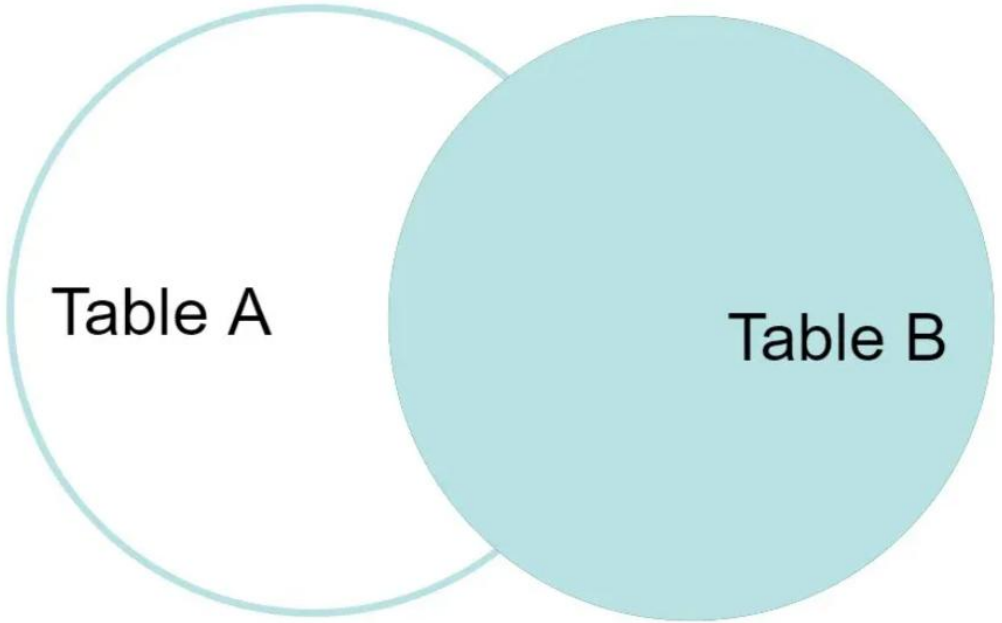
StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
RAVI	NULL

SQL RIGHT JOIN

RIGHT JOIN returns all the rows of the table on the right side of the join and matching rows for the table on the left side of the join. It is very similar to LEFT JOIN for the rows for which there is no matching row on the left side, the result-set will contain null. RIGHT JOIN is also known as RIGHT OUTER JOIN.



Right JOIN

SQL RIGHT JOIN

Student

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
4	5

```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
RIGHT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

SQL RIGHT JOIN

Student

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18

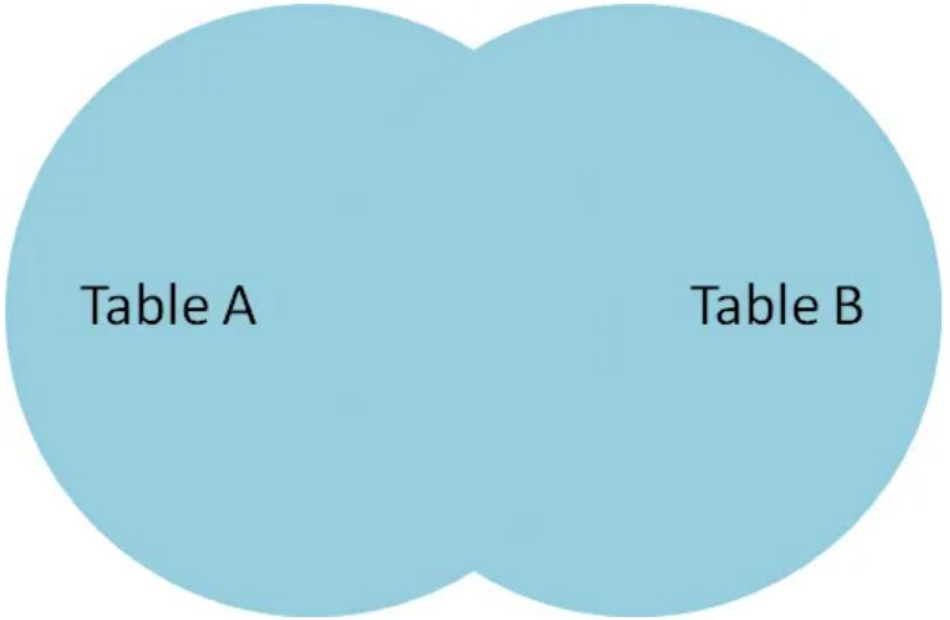
StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
NULL	4

SQL FULL JOIN

FULL JOIN creates the result-set by combining results of both LEFT JOIN and RIGHT JOIN. The result-set will contain all the rows from both tables. For the rows for which there is no matching, the result-set will contain NULL values



SQL FULL JOIN

Student

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4

```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
FULL JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

SQL FULL JOIN

Student

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3

FULL JOIN vs. CARTESIAN PRODUCT

Feature	**FULL JOIN**	**CARTESIAN PRODUCT** (CROSS JOIN)
Definition	Combines rows from both tables **based on a matching condition**.	Combines **every row** from the first table with **every row** from the second table.
Join Condition	Requires a join condition (e.g., ON A.id = B.id).	**No condition** is required.
Result Rows	Returns **matched rows** + **unmatched rows** (with NULLs).	Returns **all possible combinations** of rows.
Row Count	At most: rows in A + rows in B (if no matches).	Exactly: rows in A × rows in B.
NULLs in Result	Yes, for non-matching rows.	No NULLs are introduced unless they exist in data.
Use Case	When you want **all data** from both tables and see what matches.	Rare, used for **testing**, **combinations**, or **generating datasets**.

PRACTICE PROBLEM

Student Relation

ID	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

Takes Relation

ID	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	*null*

student natural join *takes ?*

student natural join *takes*

ID	*name*	*dept_name*	*tot_cred*	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2017	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2017	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2017	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2017	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2018	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2017	A
19991	Brandt	History	80	HIS-351	1	Spring	2018	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2018	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2017	B-
45678	Levy	Physics	46	CS-101	1	Fall	2017	F
45678	Levy	Physics	46	CS-101	1	Spring	2018	B+
45678	Levy	Physics	46	CS-319	1	Spring	2018	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2017	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2017	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2018	A-
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2017	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2018	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2017	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2017	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2018	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2017	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2018	*null*

Outer Join Examples

- Relation *course*

course_id	*title*	*dept_name*	*credits*
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

- Relation *prereq*

course_id	*prereq_id*
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

- *course* **natural left outer join** *prereq ?*
- *course* **natural right outer join** *prereq?*
- *course* **natural full outer join** *prereq ?*

Left Outer Join

- *course* **natural left outer join** *prereq*

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*null*

  - In relational algebra: *course* ⟕ *prereq*

Right Outer Join

- *course* **natural right outer join** *prereq*

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	*null*	*null*	*null*	CS-101

  - In relational algebra: *course* ⟖ *prereq*

Full Outer Join

- *course* **natural full outer join** *prereq*

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*null*
CS-347	*null*	*null*	*null*	CS-101

- In relational algebra: *course* ⟗ *prereq*

Joined Types and Conditions

- **Join operations** take two relations and return as a result another relation.
- These additional operations are typically used as subquery expressions in the **from** clause
- **Join condition** – defines which tuples in the two relations match.
- **Join type** – defines how tuples in each relation that do not match any tuple in the other relation (based on the join condition) are treated.

Join types
inner join
left outer join
right outer join
full outer join

Join conditions
natural
on <predicate>
using $(A_1, A_2, \ldots, A_n)$

Views

- In some cases, it is not desirable for all users to see the entire logical model (that is, all the actual relations stored in the database.)
- Consider a person who needs to know an instructors name and department, but not the salary. This person should see a relation described, in SQL, by

  select *ID*, *name*, *dept_name*
  from *instructor*

- A **view** provides a mechanism to hide certain data from the view of certain users.
- Any relation that is not of the conceptual model but is made visible to a user as a "virtual relation" is called a **view**.

View Definition

- A view is defined using the **create view** statement which has the form

  create view *v* **as** < query expression >

  where <query expression> is any legal SQL expression. The view name is represented by *v*.
- Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.
- View definition is not the same as creating a new relation by evaluating the query expression
  - Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.

View Definition and Use

- A view of instructors without their salary

```
create view faculty as
        select ID, name, dept_name
        from instructor
```

- Find all instructors in the Biology department

```
select name
from faculty
where dept_name = 'Biology'
```

- Create a view of department salary totals

```
create view departments_total_salary(dept_name, total_salary) as
    select dept_name, sum (salary)
    from instructor
    group by dept_name;
```

THANK YOU

